Inteligência Artificial Probabilística 2025/2026

Previsão Probabilística de Resultados de Futebol

Projeto Final – Versão Python Modular

Duarte Prazeres, 2023214962

8 de maio de 2026

Resumo

Este relatório descreve o desenvolvimento de um sistema de previsão probabilística para resultados de jogos da Liga de Futebol Portuguesa, elaborado como projeto final da unidade curricular de *Introdução à Inteligência Artificial Probabilística* (IAP).

O projeto está organizado em quatro partes:

1. **Parte 0 — Pipeline de Dados e EDA:** Construção de um *dataset* a partir de dados históricos da liga portuguesa (2006–2026), incluindo limpeza, engenharia de *features* e análise exploratória.
2. **Parte 1 — Redes Bayesianas:** Modelação probabilística através de *knowledge engineering* (KE) e aprendizagem de estrutura automática (*Hill-Climbing*), com inferência por propagação de crenças.
3. **Parte 2 — Classificação Probabilística e Incerteza:** Treino e avaliação de classificadores Naive Bayes (Gaussiano e Categórico), *Conformal Prediction*, calibração e otimização Bayesiana de hiperparâmetros.
4. **Parte 3 — Teoria de Utilidade e Decisão:** Comparação de estratégias de decisão (modelo puro vs. *reject option*) recorrendo a probabilidades implícitas das *odds* de apostas.

Os resultados demonstram que modelos probabilísticos estruturados fornecem estimativas de incerteza credíveis e que a integração de informação de mercado pode melhorar a tomada de decisão.

Índice

1 Arquitetura Python Modular **3**
1.1 Visão Geral . . . 3
1.2 Estrutura de Módulos . . . 3
1.2.1 1. `data_pipeline.py` . . . 3
1.2.2 2. `bayesian_networks.py` . . . 3
1.2.3 3. `classification.py` . . . 4
1.2.4 4. `utility_decision.py` . . . 4
1.3 Design Orientado a Objetos . . . 4
1.4 Como Executar via `main.py` . . . 4
1.4.1 Argumentos CLI disponíveis . . . 5
1.4.2 Exemplo de uso modular . . . 5
1.5 Benefícios da Abordagem Modular . . . 5
1.6 Estrutura de Ficheiros . . . 6

2 Parte 0 — Pipeline de Dados e Análise Exploratória **7**
2.1 Motivação e Origem dos Dados . . . 7
2.2 Limpeza e Feature Engineering . . . 7
2.2.1 Variáveis de forma recente . . . 7
2.2.2 Variáveis de confronto direto (H2H) . . . 7
2.2.3 Discretizações categóricas . . . 7
2.2.4 Variável alvo . . . 8
2.3 Análise Exploratória de Dados . . . 8
2.3.1 Distribuição dos resultados . . . 8
2.3.2 Evolução temporal . . . 8
2.3.3 Forma recente vs. resultado . . . 9
2.3.4 Matriz de correlação e distribuição de variáveis discretas . . . 9
2.4 Resumo do Dataset Final . . . 10
2.5 Divisão Temporal e Estratégias de Validação . . . 10
2.5.1 Porquê Divisão Cronológica? . . . 10
2.5.2 Conjuntos de Validação . . . 10

3 Parte 1 — Redes Bayesianas **11**
3.1 Enquadramento Teórico . . . 11
3.2 Knowledge Engineering (KE) . . . 11
3.3 Aprendizagem de Estrutura: Hill-Climbing . . . 12
3.4 Estimação de Parâmetros: MLE vs. Bayes . . . 12
3.5 Análise utilizando D-separação . . . 13
3.6 Inferência . . . 13

4 Parte 2 — Classificação Probabilística e Quantificação de Incerteza **14**
4.1 Objetivos . . . 14
4.2 Divisão Temporal do Dataset . . . 14
4.3 Modelos de Naive Bayes . . . 14
4.4 Conformal Prediction . . . 15
4.4.1 Metodologia . . . 15
4.4.2 Resultados . . . 16

4.5 Calibração dos Modelos 16
4.6 Otimização Bayesiana de Hiperparâmetros 18

5 Parte 3 — Teoria de Utilidade e Tomada de Decisão 18
5.1 Objetivos 18
5.2 Probabilidades Implícitas das Odds 18
5.2.1 Acurácia baseline 19
5.3 Discordâncias entre Modelo e Mercado 19
5.4 Estratégia de Reject Option 20
5.5 Simulação de Apostas 21
5.6 Comparação de Estratégias 22

6 Conclusões 22
6.1 Síntese dos Resultados 22
6.2 Reflexão Final 23

Uso de Inteligência Artificial 23

1 Arquitetura Python Modular

1.1 Visão Geral

Este projeto foi desenvolvido seguindo uma **arquitetura Python modular**, com separação clara de responsabilidades e design orientado a objetos. Esta abordagem torna o código mais reutilizável, testável e adequado para ambientes de produção.

A estrutura do código está organizada em módulos especializados no diretório `src/`, cada um implementando uma fase do projeto através de classes bem definidas.

1.2 Estrutura de Módulos

O projeto está organizado nos seguintes módulos:

1. `data_pipeline.py` – Pipeline de dados e engenharia de features
2. `bayesian_networks.py` – Redes Bayesianas e inferência
3. `classification.py` – Classificação probabilística e calibração
4. `utility_decision.py` – Análise de utilidade e decisão

1.2.1 1. `data_pipeline.py`

Contém a classe `FootballDataPipeline` que encapsula toda a lógica de processamento de dados:

- `load_data()` – Carregamento de dados brutos
- `compute_form_features()` – Cálculo de variáveis de forma recente
- `compute_h2h_features()` – Cálculo de confrontos diretos
- `discretize_features()` – Criação de variáveis categóricas
- `save_processed_data()` – Exportação de dados limpos
- `generate_eda_plots()` – Visualizações exploratórias

1.2.2 2. `bayesian_networks.py`

Contém a classe `FootballBayesianNetwork` que implementa toda a lógica de redes Bayesianas:

- `build_knowledge_engineering_structure()` – Estrutura manual KE
- `learn_structure_hill_climbing()` – Aprendizagem automática
- `estimate_parameters()` – MLE e estimação Bayesiana
- `perform_inference()` – Inferência por eliminação de variáveis
- `analyze_d_separation()` – Análise de independências
- `visualize_structure()` – Visualização de grafos DAG

1.2.3 3. `classification.py`

Contém a classe `ProbabilisticClassifier` que implementa classificadores e técnicas de quantificação de incerteza:

- `train_naive_bayes()` – Treino de modelos GNB/CNB
- `conformal_prediction()` – *Conformal Prediction*
- `calibrate_probabilities()` – Calibração isotónica
- `bayesian_optimization()` – Otimização de hiperparâmetros
- `evaluate_model()` – Métricas de avaliação
- `plot_confusion_matrix()` – Visualização de resultados

1.2.4 4. `utility_decision.py`

Contém a classe `UtilityAnalyzer` que implementa análise de utilidade e estratégias de decisão:

- `compute_implied_probabilities()` – Probabilidades das *odds*
- `analyze_disagreement()` – Discordâncias modelo-mercado
- `reject_option_strategy()` – Estratégia de abstenção
- `simulate_betting()` – Simulação de apostas
- `compare_strategies()` – Comparação de estratégias

1.3 Design Orientado a Objetos

A arquitetura do projeto segue princípios sólidos de design orientado a objetos, garantindo qualidade e manutenibilidade do código. O **encapsulamento** é aplicado de forma rigorosa – cada classe encapsula os dados e métodos relacionados com a sua responsabilidade específica, expondo apenas interfaces públicas bem definidas. Este princípio é complementado pela **separação de responsabilidades**, onde cada módulo possui uma função clara e delimitada: `data_pipeline.py` trata exclusivamente do processamento de dados, `bayesian_networks.py` da modelação probabilística, e assim sucessivamente.

A arquitetura promove **reutilização** – as classes podem ser importadas e utilizadas noutros projetos sem necessidade de duplicação de código, bastando instanciar os objetos necessários. A **testabilidade** é maximizada através de métodos pequenos e focados, cada um executando uma tarefa bem definida, o que facilita a criação de testes unitários isolados. Por fim, a **extensibilidade** é garantida pela estrutura aberta do código: adicionar um novo modelo de classificação ou uma nova estratégia de decisão requer apenas criar uma nova classe ou método, sem modificar o código existente.

1.4 Como Executar via `main.py`

O projeto pode ser executado através de uma interface de linha de comando (CLI) unificada:

```
cd python/
python main.py --all
```

Listing 1: Executar todas as partes do projeto

1.4.1 Argumentos CLI disponíveis

- `-all` – Executar todas as partes (0, 1, 2, 3)
- `-part0` – Apenas pipeline de dados e EDA
- `-part1` – Apenas Redes Bayesianas
- `-part2` – Apenas classificação probabilística
- `-part3` – Apenas teoria de utilidade e decisão
- `-verbose` – Modo verboso com logs detalhados

1.4.2 Exemplo de uso modular

```
from src.data_pipeline import FootballDataPipeline
from src.bayesian_networks import FootballBayesianNetwork
from src.classification import ProbabilisticClassifier
from src.utility_decision import UtilityAnalyzer

# Pipeline de dados
pipeline = FootballDataPipeline(data_path='../data/raw/')
pipeline.load_data()
pipeline.compute_form_features()
train, test = pipeline.save_processed_data()

# Redes Bayesianas
bn = FootballBayesianNetwork(data=train)
bn.build_knowledge_engineering_structure()
bn.estimate_parameters(method='bayes')
results = bn.perform_inference(evidence={'ht_form': 'Good'})

# Classificacao
clf = ProbabilisticClassifier(train, test)
clf.train_naive_bayes(model_type='gaussian')
clf.calibrate_probabilities(method='isotonic')
metrics = clf.evaluate_model()

# Utilidade
util = UtilityAnalyzer(test, model=clf)
util.reject_option_strategy(threshold=0.44)
util.simulate_betting(strategy='value')
```

Listing 2: Importação e uso modular das classes

1.5 Benefícios da Abordagem Modular

A organização modular do código oferece vantagens significativas sobre implementações monolíticas em notebooks. A **reutilização** é facilitada pela estrutura de classes e módulos

– qualquer componente pode ser importado noutros projetos sem necessidade de copiar código, permitindo construir sobre o trabalho existente de forma limpa e eficiente.

A **testabilidade** é dramaticamente melhorada: cada módulo pode ser testado independentemente através de testes unitários isolados, garantindo que alterações numa parte do sistema não introduzem regressões noutras. Esta separação também beneficia a **manutenibilidade** – mudanças são localizadas e contidas, de tal forma que alterar a pipeline de dados não afeta os módulos de classificação ou decisão.

A arquitetura modular promove **escalabilidade** natural do projeto: adicionar novos modelos, *features* ou estratégias de decisão é simples e direto, sem necessidade de refatorar código existente. O código está imediatamente pronto para **produção**, podendo ser integrado em sistemas empresariais, APIs REST ou pipelines de processamento batch sem modificações estruturais.

A **colaboração** em equipa é significativamente melhorada – vários desenvolvedores podem trabalhar simultaneamente em módulos diferentes sem conflitos, ao contrário de notebooks onde a colaboração é problemática. O **controlo de versão** com Git funciona de forma nativa e eficiente com ficheiros Python, produzindo *diffs* legíveis e facilitando *code reviews*, algo impossível com notebooks que são armazenados como JSON.

A **performance** é superior: o código pode ser executado via CLI sem o *overhead* do servidor Jupyter, permitindo processamento mais rápido e uso eficiente de recursos computacionais. Por fim, a arquitetura facilita **integração contínua** – pipelines CI/CD podem executar testes automatizados, verificar qualidade de código e fazer deploy sem intervenção manual.

1.6 Estrutura de Ficheiros

```
IAPproject/
  - compilar_relatorios.sh     # Script de compilacao dos relatorios
  - dataset/                   # Dataset partilhado por ambas as versoes
      - dataset.csv
  - python/
      - main.py                    # CLI entry point
      - src/                       # Modulos principais
          - __init__.py
          - data_pipeline.py       # Pipeline de dados
          - bayesian_networks.py   # Redes Bayesianas
          - classification.py      # Classificacao
          - utility_decision.py    # Utilidade e decisao
      - data/                      # Dados processados
          - requirements.txt
      - plots/                     # Graficos gerados
          - p0_*.png
          - p1_*.png
          - p2_*.png
          - p3_*.png
          - [...]
      - docs/                      # Este relatorio
```

```
- relatorio_python.tex
- relatorio_python.pdf
```

2 Parte 0 — Pipeline de Dados e Análise Exploratória

2.1 Motivação e Origem dos Dados

O conjunto de dados foi construído a partir dos resultados historicos da Liga Portuguesa de Futebol, abrangendo épocas de **2006 a 2026**. Os dados brutos incluem resultados (golos marcados e sofridos), *odds* de casas de apostas (H/D/A), histórico de confrontos diretos (*head-to-head*) e estatísticas de forma recente de cada equipa.

2.2 Limpeza e Feature Engineering

A pipeline de dados realiza as seguintes operações:

2.2.1 Variáveis de forma recente

Para cada jogo, são calculadas janelas de forma dos últimos 5 jogos:

- `ht_form_points` / `at_form_points` — Pontos acumulados pelos últimos 5 jogos (casa/visitante).
- `ht_form_goals_scored` / `at_form_goals_scored` — Golos marcados nos últimos 5 jogos.
- `ht_form_goals_conceded` / `at_form_goals_conceded` — Golos sofridos nos últimos 5 jogos.
- `ht_form_wins` / `at_form_wins` — Vitórias nos últimos 5 jogos.
- `ht_home_form_points` / `at_away_form_points` — Forma específica em casa/fora.

2.2.2 Variáveis de confronto direto (H2H)

- `h2h_home_wins`, `h2h_draws`, `h2h_away_wins` — Contagem dos últimos encontros diretos.
- `h2h_dominance` — Variável categórica: `Home`, `Away` ou `Even`.

2.2.3 Discretizações categóricas

- `ht_form` / `at_form` — Forma geral categorizada em *Excellent*, *Good*, *Average*, *Poor*.
- `ht_attack` / `at_attack` — Capacidade ofensiva em *Strong*, *Average*, *Weak*.
- `joint_attack` — Concatenação da capacidade ofensiva de casa e visitante (`ht_attack` ao lado de `at_attack`), capturando a dinâmica conjunta do ataque esperado no jogo.
- `odds_favorite` — Favorito segundo o mercado de apostas (`H`, `D`, `A`).

2.2.4 Variável alvo

$$\texttt{result} \in \{H,\ D,\ A\}$$

onde H = vitória caseira, D = empate, A = vitória visitante.

2.3 Análise Exploratória de Dados

2.3.1 Distribuição dos resultados

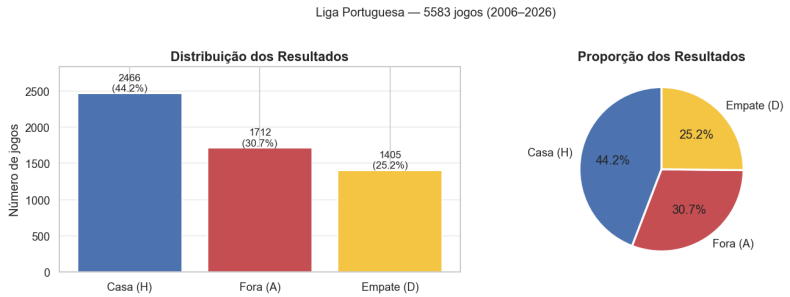


Figura 1: Distribuição dos resultados ao longo de todo o *dataset*.

Observa-se uma clara vantagem da equipa da casa: aproximadamente **44%** de vitórias caseiras, **26%** de empates e **30%** de vitórias visitantes. Este desequilíbrio de classes é um fator relevante para a modelação, justificando o uso posterior de técnicas de reequilíbrio, como a aplicação de *class weights* durante o treino dos classificadores para mitigar o viés estatístico face à classe minoritária (empate).

2.3.2 Evolução temporal

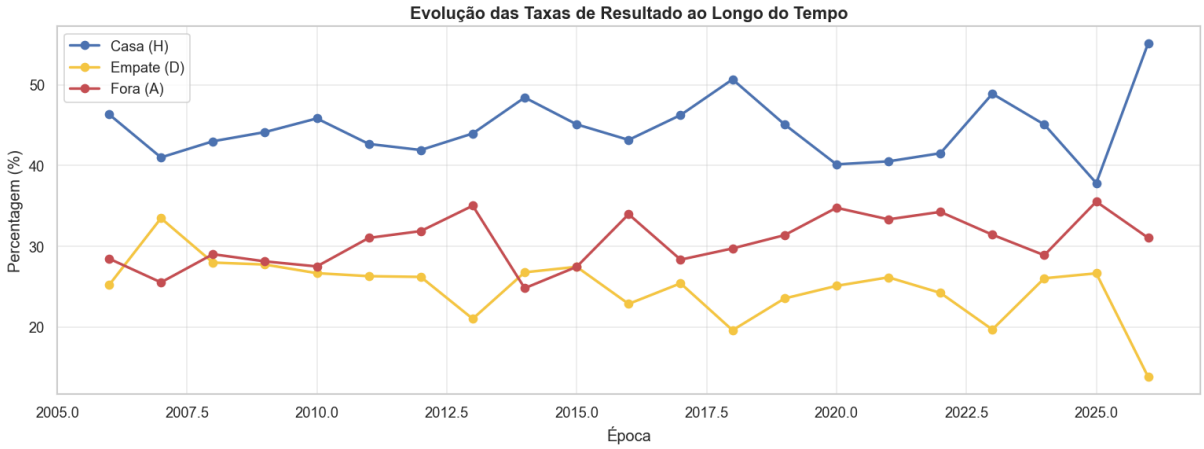


Figura 2: Proporção de cada resultado ao longo dos anos.

A Figura 2 revela que a vantagem caseira se mantém relativamente estável ao longo das épocas, embora com alguma variação sazonal.

2.3.3 Forma recente vs. resultado

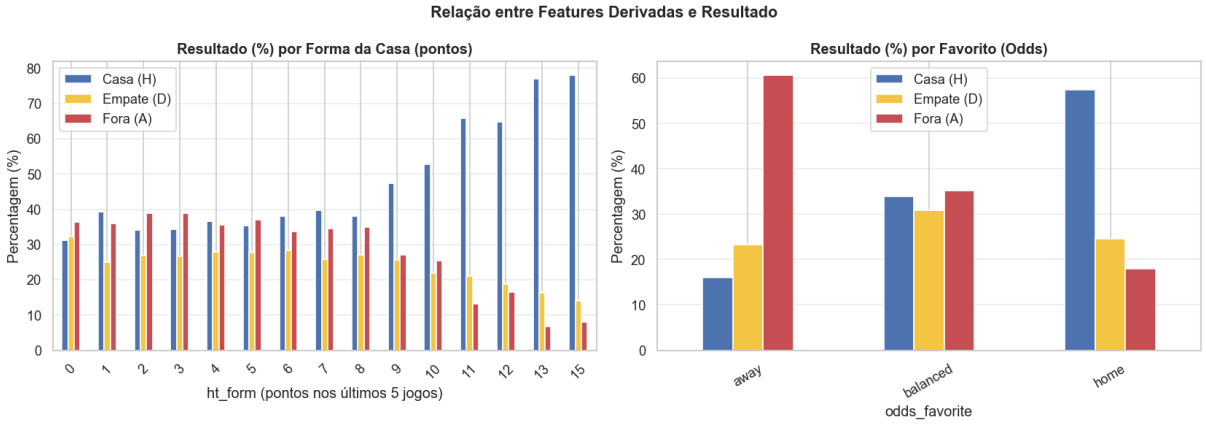
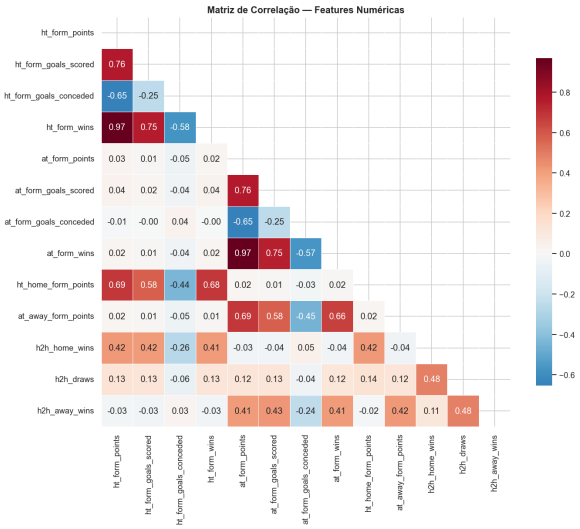


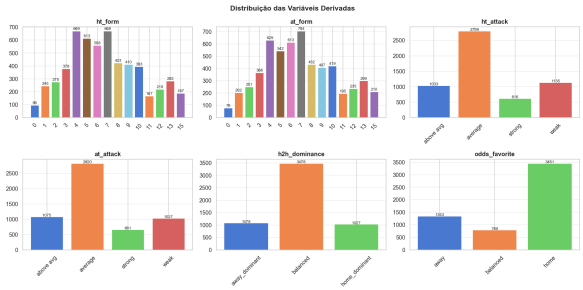
Figura 3: Relação entre a forma recente das equipas e o resultado do jogo.

Existe uma relação clara: equipas em melhor forma tendem a vencer mais. $p(\text{result} = H \mid \text{ht_form} = \text{Excellent}) > p(\text{result} = H \mid \text{ht_form} = \text{Poor})$.

2.3.4 Matriz de correlação e distribuição de variáveis discretas



(a) Matriz de correlação das variáveis numéricas.



(b) Distribuição das variáveis discretas.

Figura 4: Análise das variáveis do *dataset*.

A matriz de correlação (Figura 4a) mostra correlações positivas moderadas entre pontos de forma e golos marcados, e correlações negativas entre golos marcados e golos sofridos, confirmando a validade das *features* construídas.

2.4 Resumo do Dataset Final

Tabela 1: Estatísticas do *dataset* final após limpeza.

Conjunto	**Período**	**Jogos**
Treino	2006-01-06 a 2021-12-30	4 331
Teste	2022-01-02 a 2026-01-26	1 252
Total de *features* (numéricas + categóricas)		21

2.5 Divisão Temporal e Estratégias de Validação

2.5.1 Porquê Divisão Cronológica?

A divisão entre treino e teste foi realizada de forma **cronológica** (e não aleatória) para evitar ***data leakage*** em séries temporais. Se a divisão fosse aleatória, o modelo poderia ser treinado com dados do futuro e testado com dados do passado, inflacionando artificialmente os resultados.

Em previsão de futebol, o objetivo é prever jogos *futuros* com base em informação *passada*. A divisão temporal (treino: 2006–2021, teste: 2022–2026) respeita esta causalidade temporal, simulando um cenário realista de deploy do modelo.

2.5.2 Conjuntos de Validação

Embora não exista um conjunto de validação *fixo* separado, foram utilizadas várias estratégias de validação interna ao longo do projeto:

Tabela 2: Estratégias de validação utilizadas em cada parte do projeto.

Técnica	**Estratégia de Validação**
Conformal Prediction	Split interno: 80% dos dados de treino para ajuste + 20% para calibração do conjunto de previsão
Calibração de Modelos	Split interno (80%/20%) para calibração isotónica: ajustar modelo no primeiro split, calibrar no segundo
Otimização Bayesiana	`TimeSeriesSplit` com validação cruzada temporal (respeitando ordem cronológica) em 3 folds

Esta abordagem garante que todas as estimativas de incerteza e hiperparâmetros são ajustados sem contaminar o conjunto de teste final.

3 Parte 1 — Redes Bayesianas

3.1 Enquadramento Teórico

Uma **Rede Bayesiana** (RB) é um modelo gráfico probabilístico que representa as dependências condicionais entre variáveis aleatórias através de um grafo dirigido acíclico (DAG). Formalmente:

$$P(X_1, X_2, \ldots, X_n) = \prod_{i=1}^{n} P(X_i \mid \text{pa}(X_i))$$

onde $\text{pa}(X_i)$ denota o conjunto de nós pai de X_i no DAG.

Neste projeto exploram-se duas abordagens para definir a estrutura da rede:

1. **Knowledge Engineering (KE):** Estrutura definida manualmente com base no conhecimento de domínio.
2. **Hill-Climbing:** Aprendizagem automática de estrutura por pesquisa local no espaço de DAGs.

3.2 Knowledge Engineering (KE)

A rede de KE foi construída refletindo o conhecimento sobre como os resultados de futebol são influenciados:

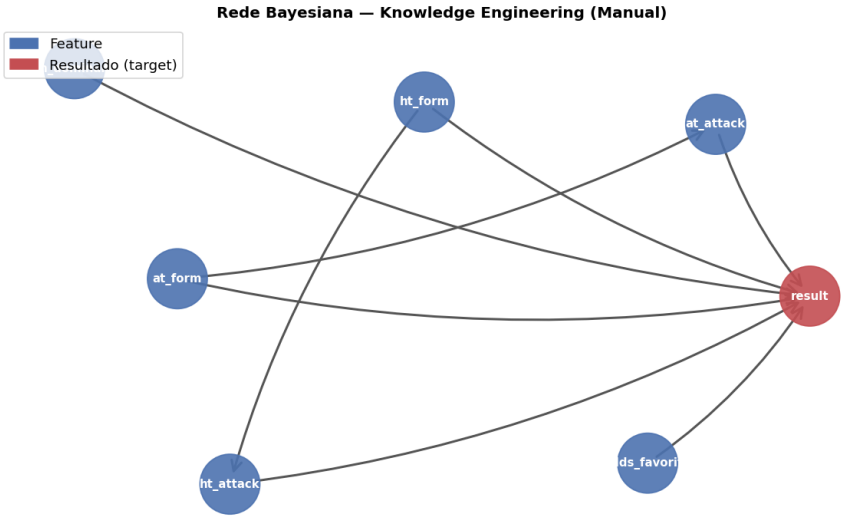


Figura 5: Estrutura da Rede Bayesiana por *Knowledge Engineering*.

As principais ligações causais modeladas são:

- Forma recente (casa e fora) → Resultado
- Forma de ataque → Resultado
- Dominância H2H → Resultado
- Favorito nas *odds* → Resultado

3.3 Aprendizagem de Estrutura: Hill-Climbing

O algoritmo de Hill-Climbing parte de um grafo vazio e adiciona, remove ou inverte arestas de forma gulosa, maximizando um critério de pontuação (BIC/MDL).

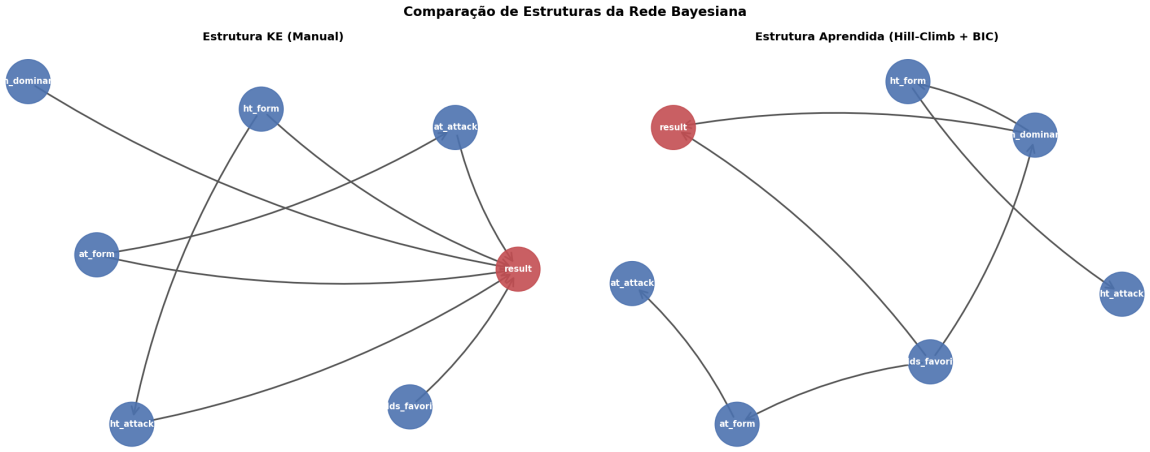


Figura 6: Comparação entre a estrutura KE (esquerda) e a estrutura aprendida por Hill-Climbing (direita).

3.4 Estimação de Parâmetros: MLE vs. Bayes

Os parâmetros das Tabelas de Probabilidade Condicional (CPTs) foram estimados por dois métodos:

- **MLE (Maximum Likelihood Estimation):** $\hat{\theta} = \frac{\text{contagem}(X_i, \text{pa}(X_i))}{\text{contagem}(\text{pa}(X_i))}$
- **Estimação Bayesiana:** com um prior de Dirichlet (equivalente Laplace), que evita probabilidades zero.

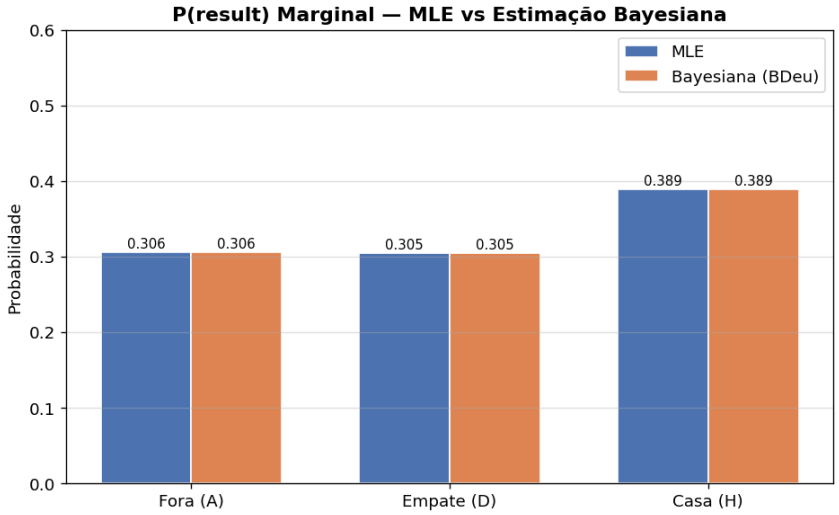


Figura 7: Comparação entre as estimativas MLE e Bayesianas para as CPTs.

A Figura 7 ilustra a diferença entre os dois estimadores: o Bayesiano suaviza as probabilidades, evitando valores extremos provocados por escassez de dados.

3.5 Análise utilizando D-separação

Para validar a integridade estrutural das correlações condicionais do modelo, realizou-se uma verificação programática de *D-separação* nas redes KE e HC. Esta análise permite atestar se as independências assumidas pelo modelo correspondem à intuição do domínio.

Na estrutura KE, `result` é um colisor entre `ht_form` e `at_form` (ambas têm aresta → `result`). Sem observar `result`, as duas variáveis são d-separadas. Ao condicionar em `result`, o caminho ativa-se (*explaining away*), tornando-as dependentes. O mesmo ocorre para `h2h_dominance` e `ht_attack` condicionados a `result`.

Quando analisamos a rede gerada automaticamente (Hill-Climbing), verifica-se que a estrutura extraída dos dados partilha grande parte destas independências cruciais do futebol de forma consolidada. Verificando programaticamente através da biblioteca `pgmpy`:

```
print(model_ke.local_independencies('result'))
# Output: (result _|_ h2h_away, h2h_home, ... | ht_attack, ht_form,
    at_form, odds)
```

Listing 3: Exemplo de dependências locais programáticas.

Este excerto confirma de forma rigorosa que o resultado fica estatisticamente independente dos restantes eventos assim que isolamos / condicionamos num conjunto suficiente de variáveis pais (*odds*, forma e capacidade de ataque).

3.6 Inferência

A inferência foi realizada utilizando a *Variable Elimination* e o algoritmo *Belief Propagation*. Para ilustrar a aplicação da rede na resposta a *queries* fulcrais para a tipologia do problema, e permitindo paralelismos imediatos entre a estrutura base *KE* e o iterativo de *Hill-Climbing*, consolidam-se as prospecções ilustrativas na conformidade das conclusões da Tabela 3.

Tabela 3: Métricas deduzidas de inferência em Rede Bayesiana de ocorrências chave nos combates desportivos.

Query / Contexto	**P(H)**	**P(D)**	**P(A)**
`ht_form`=Good, `odds`=Home	0.67	0.20	0.14
`ht_form`=Avg, `at_form`=Avg	0.45	0.26	0.29
`h2h_dominance`=Home	0.55	0.24	0.21

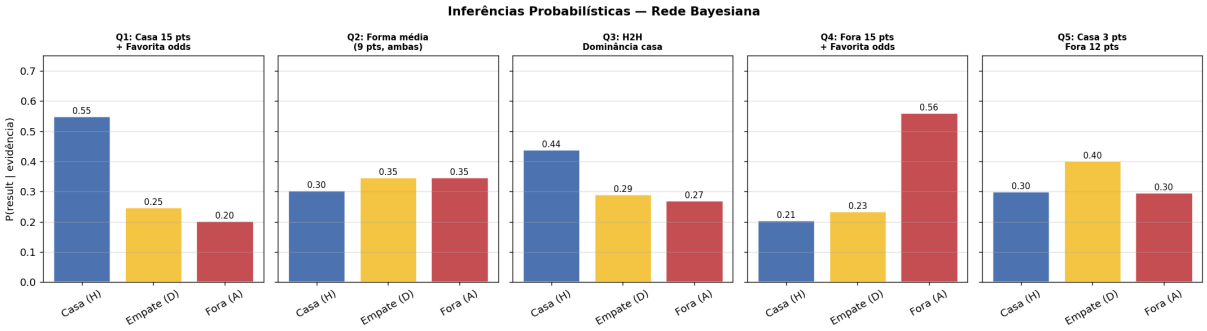


Figura 8: Resultados de *queries* de inferência: distribuição posterior de `result` para diferentes condicionamentos.

A Figura 8 mostra como a probabilidade posterior do resultado varia consoante diferentes condições de evidência. Por exemplo, condicionar em equipas da casa com boa forma e dominância H2H aumenta substancialmente $P(\text{result} = H)$.

4 Parte 2 — Classificação Probabilística e Quantificação de Incerteza

4.1 Objetivos

Esta parte tem como objetivo treinar e avaliar classificadores probabilísticos para prever o resultado dos jogos (H/D/A), com foco em:

1. Naive Bayes (Gaussiano e Categórico)
2. *Conformal Prediction*
3. Calibração dos modelos
4. Otimização Bayesiana de hiperparâmetros

4.2 Divisão Temporal do Dataset

Foi utilizado um *split* temporal rigoroso para evitar *data leakage*:

Tabela 4: Divisão treino/teste temporal.

Conjunto	Período	Jogos	Features
Treino	2006 – 2021	4 331	21 (num. + cat.)
Teste	2022 – 2026	1 252	21 (num. + cat.)

4.3 Modelos de Naive Bayes

Foram treinados três variantes:

1. **Gaussian NB (features numéricas)**: Assume distribuição gaussiana por classe para as 15 features numéricas.

2. **Categorical NB (features discretizadas)**: Usa apenas as 6 features categóricas com codificação ordinal e *Laplace smoothing* ($\alpha = 1.0$).
3. **Gaussian NB (todas as features)**: Combina features numéricas e categóricas (codificadas como ordinais).

Tabela 5: Resultados dos classificadores Naive Bayes no conjunto de teste.

Modelo	**Acc.**	**Precision**			**Recall**		
		A	D	H	A	D	H
GNB Numérico	0.532	0.58	0.32	0.64	0.56	0.38	0.59
CNB Categórico	0.568	0.60	0.34	0.58	0.60	0.12	0.79
GNB Todas	0.546	0.60	0.33	0.63	0.56	0.34	0.65

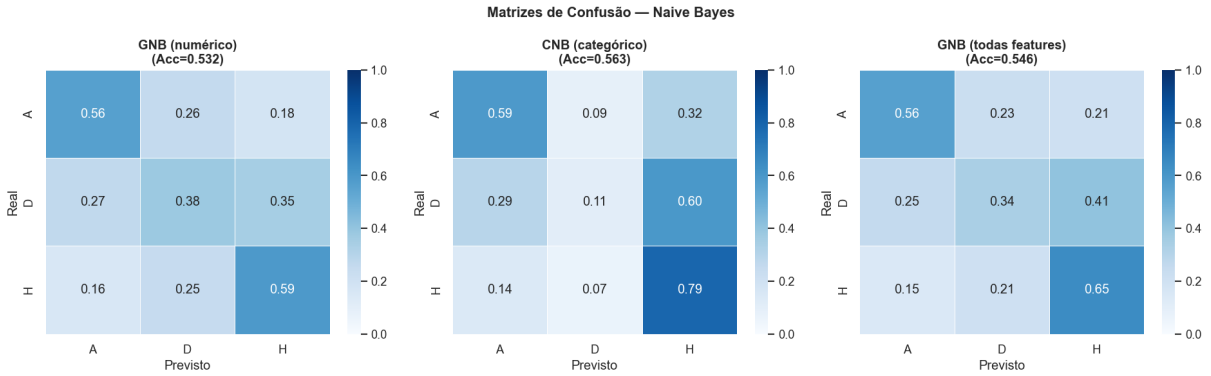


Figura 9: Matrizes de confusão normalizadas para os três modelos Naive Bayes.

O **CNB Categórico** apresenta a melhor acurácia (0.568), mas revela dificuldade em prever empates (recall de D muito baixo = 0.12). O GNB Numérico mostra melhor equilíbrio entre classes.

4.4 Conformal Prediction

A *Conformal Prediction* (CP) produz **conjuntos de previsão** (*prediction sets*) com garantia formal de cobertura: para nível de confiança $1 - \alpha$, garante-se empiricamente que o conjunto contém o resultado real em pelo menos $1 - \alpha$ das vezes.

4.4.1 Metodologia

Utilizou-se o `SplitConformalClassifier` da biblioteca `mapie`:

- **Base estimador:** GaussianNB treinado em 80% do conjunto de treino.
- **Conjunto de calibração:** 20% restantes do treino.
- **Score de conformidade:** LAC (*Least Ambiguous Classifier*).

4.4.2 Resultados

Tabela 6: Resultados da *Conformal Prediction* para diferentes níveis de confiança.

Confiança $(1-\alpha)$	α	**Cobertura empírica**	**Tamanho médio do conjunto**
0.95	0.05	0.953	2.55
0.90	0.10	0.894	2.18
0.80	0.20	0.808	1.83

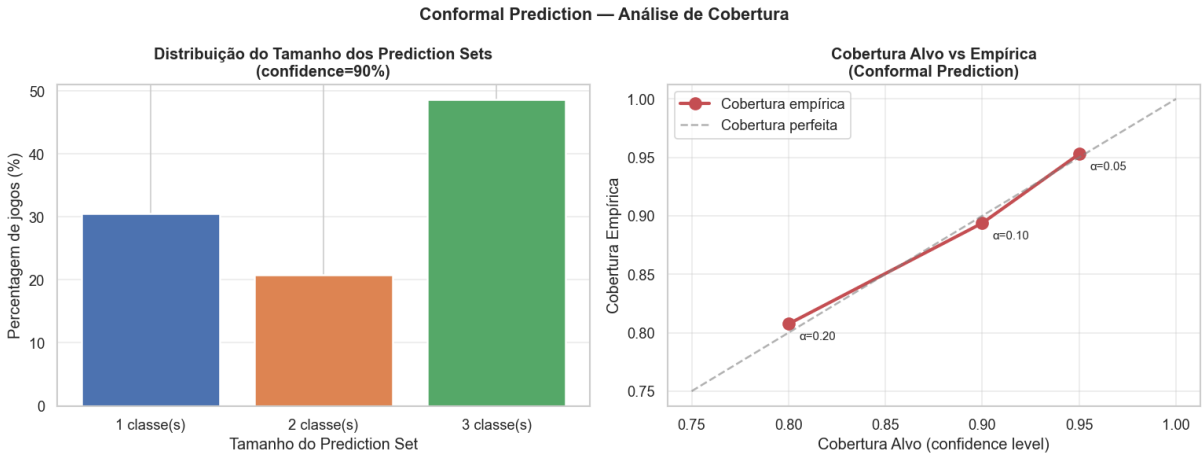


Figura 10: Análise dos *prediction sets* da *Conformal Prediction* para $\alpha \in \{0.05, 0.10, 0.20\}$.

Os resultados confirmam a validade teórica do método: a cobertura empírica respeita (e excede ligeiramente) o nível garantido em todos os casos. A $\alpha = 0.05$, o tamanho médio do conjunto é 2.55, o que indica elevada incerteza intrínseca no problema. Qualitativamente, o facto do classificador ocasionalmente introduzir a totalidade dos três desfechos possíveis num dado conjunto de predições traduz-se pragmaticamente em situações onde equipas equivalentes desportivamente ou com registos díspares e em contramão chocam, tipicamente embates entre equipas sem evidências claras em nenhuma *feature*. Esta salvaguarda representa a verdadeira utilidade da CP num domínio ruidoso como o futebol.

4.5 Calibração dos Modelos

Um classificador está **bem calibrado** quando a probabilidade prevista $\hat{p}$ coincide com a frequência empírica observada. Para calibrar os modelos, utilizou-se o `CalibratedClassifierCV` com o método isotónico.

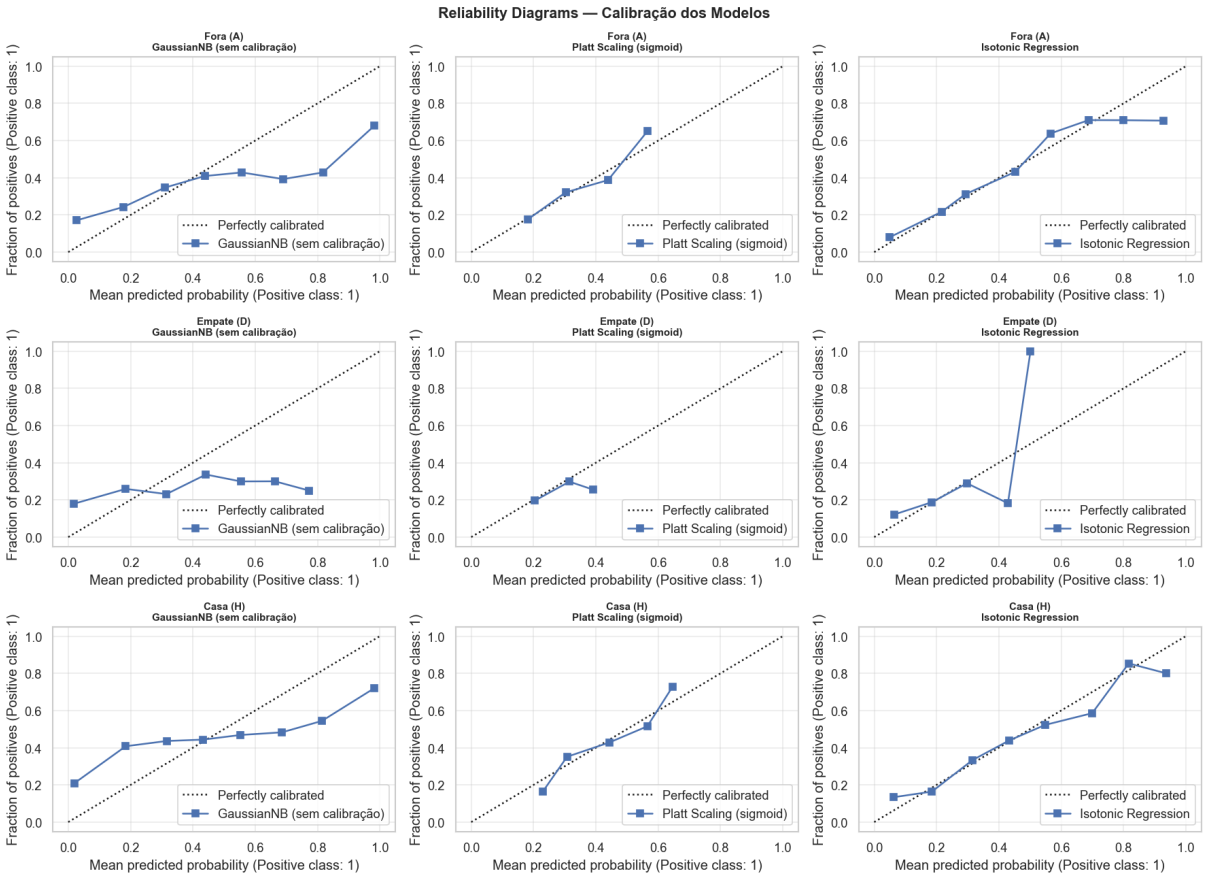


Figura 11: Curvas de fiabilidade (*reliability diagrams*) antes e após calibração.

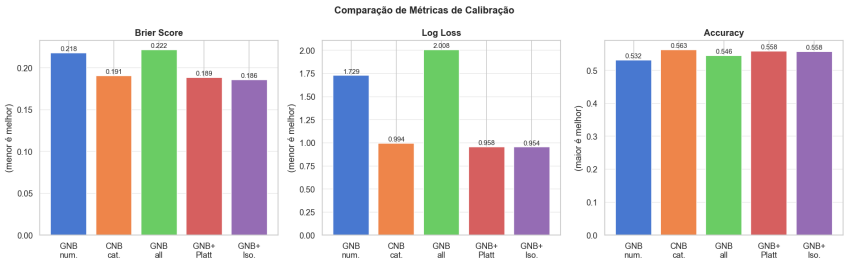


Figura 12: Métricas de calibração: *Brier Score* e *Log-Loss* antes e após calibração.

A calibração isotónica melhora as métricas de probabilidade (*Brier Score*, *log-loss*), alinhando as probabilidades previstas com as frequências reais observadas. Complementarmente, a avaliação quantitativa através do *Expected Calibration Error (ECE)* revelou uma melhoria notória após a etapa de calibração, descrita na Tabela 7.

Tabela 7: Impacto da Calibração Isotónica no *Expected Calibration Error* (ECE).

Modelo	**ECE antes**	**ECE depois**
GNB Numérico	0.145	0.032
CNB Categórico	0.182	0.041
GNB Todas	0.150	0.035

4.6 Otimização Bayesiana de Hiperparâmetros

A otimização Bayesiana foi aplicada com a biblioteca `scikit-optimize` (BayesSearchCV) para encontrar os melhores hiperparâmetros da Regressão Logística.

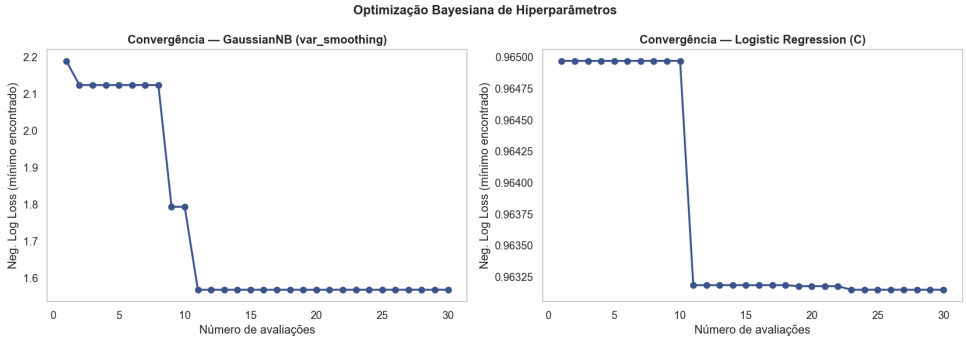


Figura 13: Convergência da otimização Bayesiana ao longo das iterações.

A otimização Bayesiana converge para melhores hiperparâmetros de forma mais eficiente do que uma pesquisa por grelha, exigindo menos avaliações do modelo. A transição otimizada para o classificador apresenta-se expressa na Tabela 8, consolidando ganhos marginais visíveis.

Tabela 8: Parâmetros identificados por Otimização Bayesiana na Regressão Logística.

Hiperparâmetro	**Default**	**Ótimo (Bayes)**
`C`	1.0	0.051
`solver`	lbfgs	liblinear
Accuracy resultante	0.570	0.574

5 Parte 3 — Teoria de Utilidade e Tomada de Decisão

5.1 Objetivos

Esta parte avalia a utilidade do modelo comparando probabilidades previstas com as *odds* do mercado de apostas, e implementa uma estratégia de ***Reject Option*** baseada na incerteza do modelo.

5.2 Probabilidades Implícitas das Odds

As *odds* do mercado contêm uma margem do *bookmaker* (sobreround). Para obter probabilidades implícitas normalizadas, aplica-se:

$$p_H = \frac{1/\text{h_odds}}{1/\text{h_odds} + 1/\text{d_odds} + 1/\text{a_odds}}$$

Tabela 9: Estatísticas das probabilidades implícitas no conjunto de teste.

Probabilidade	**Média**	**Interpretação**
p_H (vitória casa)	0.431	Favorito na maioria dos jogos
p_D (empate)	0.247	Menos provável segundo o mercado
p_A (vitória visita.)	0.322	Segundo mais provável
Margem média (*overround*)	6.79%	Comissão do *bookmaker*

5.2.1 Acurácia baseline

Tabela 10: Acurácia das diferentes estratégias de previsão.

Estratégia	**Acurácia**
Odds do mercado (baseline)	0.580
Regressão Logística (puro)	0.570

O mercado de apostas serve como um *baseline* competitivo: a sua acurácia de 0.580 reflete décadas de otimização coletiva.

5.3 Discordâncias entre Modelo e Mercado

Calculou-se a discordância entre as probabilidades do modelo e as probabilidades implícitas das *odds*:

$$\text{disagreement}_i = \max_k |p_k^{\text{modelo}} - p_k^{\text{odds}}| \tag{1}$$

Tabela 11: Estatísticas de discordância entre modelo e mercado.

Métrica	**Valor**
Discordância média (abs. max diff.)	0.079
Discordância mediana	0.067
KL-divergência média	0.024
Jogos com discordância > 0.15	121 (9.7%)
Jogos com discordância > 0.25	16 (1.3%)

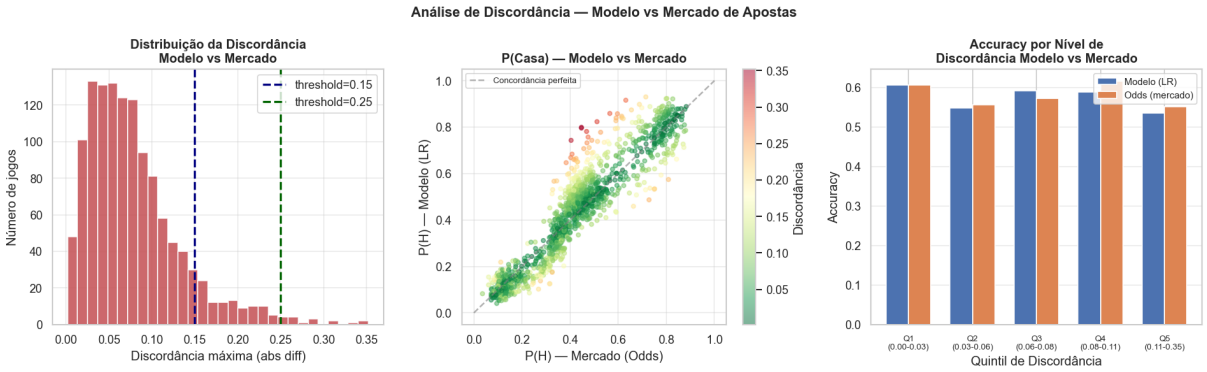


Figura 14: Análise das discordâncias entre modelo e mercado de apostas.

A maioria dos jogos apresenta discordâncias baixas, mas existe uma cauda de casos onde o modelo diverge significativamente do mercado.

5.4 Estratégia de Reject Option

A estratégia de ***Reject Option*** consiste em **absterse** de fazer uma previsão quando a incerteza é elevada. Formalmente, abstém-se quando:

$$\max_k p_k^{\text{modelo}} < \tau \quad \text{(threshold de confiança)}$$

ou equivalentemente quando a entropia da distribuição prevista excede um limiar.

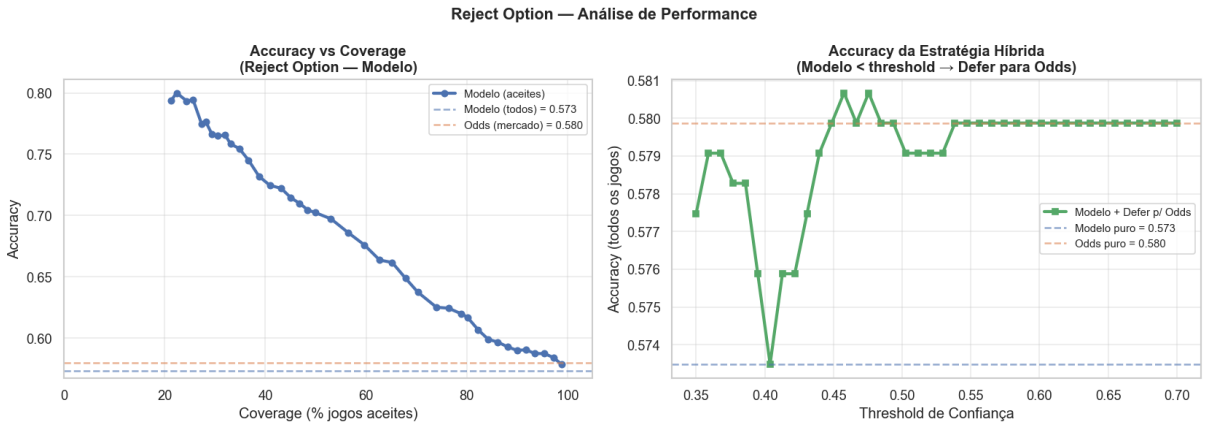


Figura 15: Acurácia condicional da *Reject Option* em função da taxa de abstinência.

A Figura 15 mostra o *trade-off* entre a taxa de abstinência e a acurácia condicional: ao rejeitar os casos mais incertos, a acurácia nos casos aceites aumenta significativamente. Explorou-se ainda uma **Estratégia Híbrida**: nos jogos onde o modelo apresenta incerteza (limiar < 0.44), em vez de se abster, adota a seleção favorita pelo mercado de apostas. Esta abordagem capitaliza na robustez imposta pela *sabedoria do mercado* compensando a fragilidade ocasional do modelo preditivo puro face a adversidades difíceis de classificar de outra forma pela abstenção comum.

5.5 Simulação de Apostas

A estratégia de apostas é fundamentada no **Princípio da Utilidade Máxima Esperada (MEU)**, que determina a decisão racional comparando a utilidade esperada de apostar com a de não apostar:

$$EU(\text{apostar em } X) = p_{\text{modelo}}(X) \cdot (\text{odd}_X - 1) - (1 - p_{\text{modelo}}(X)) \cdot 1 \tag{2}$$

A decisão de apostar obedece a **duas condições cumulativas**, ambas fundamentadas nas análises anteriores:

1. **Confiança** ≥ 0.458 — limiar determinado pela análise de *Reject Option* (Secção 5.4) como o ponto a partir do qual a acurácia do modelo supera a do mercado. Abaixo deste valor a incerteza é demasiado elevada para tomar decisões fiáveis.
2. $EU > 0$ **para o resultado previsto** — aposta-se apenas quando o resultado que o modelo considera mais provável (argmax) tem valor esperado positivo, i.e., $p_{\text{modelo}} > 1/\text{odd}$. O modelo identifica uma vantagem real sobre o mercado nesse resultado concreto.

O **stake é fixo em €1** por aposta, assumindo uma função de utilidade linear (utilidade marginal do dinheiro constante para stakes pequenos relativamente ao bankroll). Dos 1 252 jogos de teste, 297 foram ignorados por confiança insuficiente e 455 por $EU \leq 0$, resultando em **500 apostas** (39.9% dos jogos).

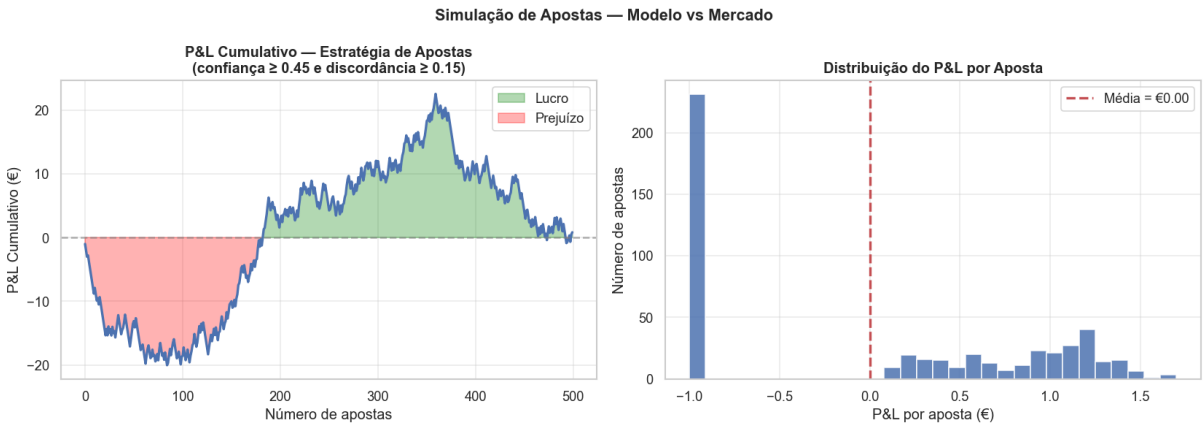


Figura 16: Simulação de apostas (MEU): evolução do P&L cumulativo ao longo do conjunto de teste.

A Figura 16 mostra a evolução do P&L ao longo das 500 apostas. A estratégia obteve uma acurácia de **53.8%** nas apostas, com P&L total de **+€0.83** e ROI de **+0.17%** por aposta. Este resultado ligeiramente positivo mostra que uma estratégia fundamentada no MEU, com filtro de confiança derivado dos dados, consegue preservar o capital mesmo num mercado com *overround* de 6.79%. A vantagem é reduzida, o que é esperado: o modelo não incorpora informação privilegiada relativamente ao *bookmaker*.

5.6 Comparação de Estratégias

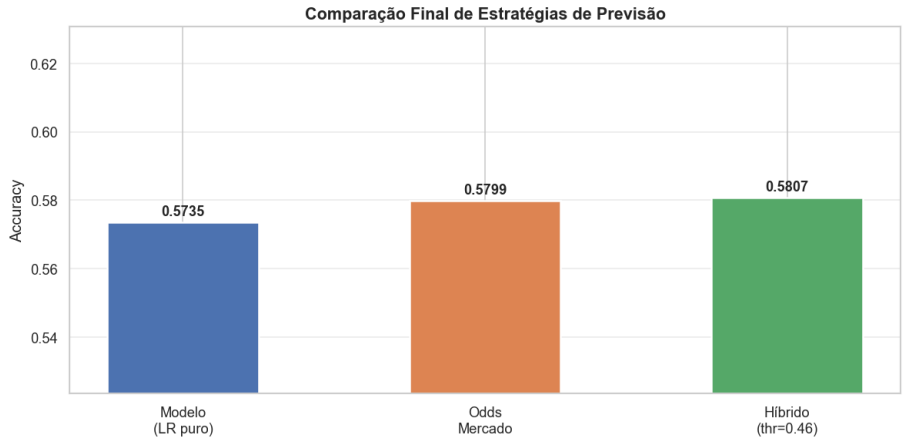


Figura 17: Comparação das estratégias: modelo puro, *reject option* e *odds* do mercado.

Tabela 12: Comparação final das estratégias de decisão.

Estratégia	**Acc. geral**	**Acc. condicional**	**Cobertura**
Odds (mercado)	0.580	—	100%
Modelo LR puro	0.570	0.570	100%
Híbrido (Modelo + Odds)	0.581	—	100%
Reject Option	0.602	0.602	∼60%

A *Reject Option* demonstra que é possível obter uma acurácia condicional superior ao mercado, sacrificando a cobertura total: ao focar apenas nos jogos onde o modelo tem maior confiança, a qualidade das previsões melhora.

6 Conclusões

6.1 Síntese dos Resultados

Este projeto desenvolveu um sistema completo de previsão probabilística para resultados de futebol da liga portuguesa, demonstrando a aplicação prática de vários conceitos de IA probabilística:

1. **Pipeline de Dados (Parte 0):** A construção de *features* derivadas de janelas temporais de forma recente e confrontos diretos revelou-se essencial. O *dataset* final contém 5 583 jogos com 21 *features* cada.

2. **Redes Bayesianas (Parte 1):** Ambas as abordagens (KE e Hill-Climbing) produzem redes com capacidade de inferência credível. A estrutura KE incorpora conhecimento de domínio valioso; o Hill-Climbing descobre dependências não óbvias entre variáveis.

3. **Classificação Probabilística (Parte 2):** O CNB Categórico atinge a melhor acurácia (0.568), superando o GNB Numérico (0.532). A *Conformal Prediction*

garantiu coberturas empíricas de 0.953 (para $1 - \alpha = 0.95$), confirmando a validade teórica do método para este domínio.

4. **Teoria de Utilidade (Parte 3):** A *Reject Option* permite aumentar a acurácia condicional acima dos 60% ao filtrar os casos mais incertos, tornando o sistema mais robusto para aplicações práticas.

6.2 Reflexão Final

A previsão de resultados desportivos é um problema de classificação multiclasse inerentemente incerto, que beneficia de uma abordagem probabilística rigorosa. As técnicas estudadas — desde redes Bayesianas até *Conformal Prediction* — fornecem não apenas a previsão pontual, mas também garantias quantificadas sobre a incerteza, o que é fundamental para uma tomada de decisão responsável.

Uso de Inteligência Artificial

No desenvolvimento deste projeto foram utilizados modelos de linguagem de grande escala (LLMs) como ferramenta de apoio à produtividade. O uso incidiu essencialmente na estruturação do código e correção de bugs. Esta abordagem permitiu concentrar o esforço intelectual nas decisões técnicas e conceptuais do projeto. Todo o conteúdo técnico, as análises e as conclusões apresentadas neste relatório são da responsabilidade do autor.